

Practical:04	Implementation and Time analysis of factorial program using iterative and recursive method
__/__/2026	

Objective:

To implement factorial calculation using both iterative and recursive methods in C++, and to evaluate their execution time, memory utilization, and system-level overhead.

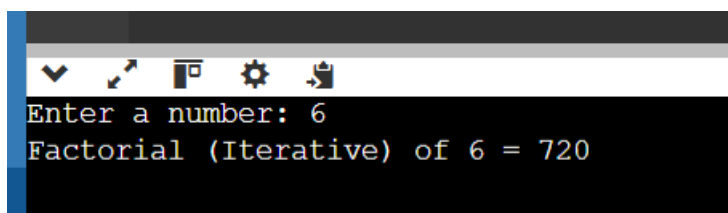
Program:**a) Iterative Method**

```
#include <iostream>

using namespace std;

long factorialIterative(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Iterative) of " << n << " = "
        << factorialIterative(n) << endl;
    return 0;
}
```

Output:A screenshot of a C++ program's output in a terminal window. The window has a dark background with a light blue title bar. The output shows the prompt "Enter a number: 6" followed by the result "Factorial (Iterative) of 6 = 720".

```
Enter a number: 6
Factorial (Iterative) of 6 = 720
```

b) Recursive Method

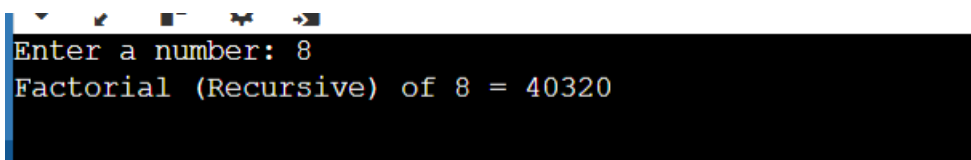
```
#include <iostream>

using namespace std;

long factorialRecursive(int n) {
    if (n <= 1)        // base case
        return 1;
    return n * factorialRecursive(n - 1); // recursive call
}

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factorial (Recursive) of " << n << " = "
        << factorialRecursive(n) << endl;
    return 0;
}
```

Output:



```
Enter a number: 8
Factorial (Recursive) of 8 = 40320
```

Analysis:

Method	Time Complexity	Space Complexity	Stack Overhead
Iterative Method	$O(n)$	$O(1)$	None
Recursive Method	$O(n)$	$O(n)$	High (proportional to n)

Conclusion:

Practical testing shows that both iterative and recursive methods compute factorials quickly for small numbers, but iteration is safer and more memory-efficient because it avoids function call stack overhead and stack overflow risks.